Discovery Executive Summary

Project: discovery12345 · **Generated:** 7/17/2026, 1:33:46 PM

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	76 / 100 — High Risk

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk cross-domain coupling (H8), legacy React Router v5 patterns (F5), and duplicated domain logic in view components (H10).

EXECUTIVE SUMMARY

This TARGET_WORKSPACE is a **frontend-only** social-network SPA (`social-media-react`) with **90 source files** (12 pages, 45 components, 11 services, 4 Redux modules). **No backend/server layer exists in the repository** — all persistence goes through an external REST API at `localhost:3030/api/` (production: `/api/`). Architectural health is **mixed**: HTTP access is partially centralized via `httpService` and domain service modules, but **presentation components bypass Redux** in 16 files, **business rules are duplicated** across post/comment/reply components, and **four feature domains** (user, post/comment, chat, activity) are tightly coupled through shared Redux state, socket orchestration in `Main.jsx`, and direct cross-service calls. The dominant risks are **legacy React Router v5 patterns (20 files)**, **cross-domain coupling (8 access points)**, and **inconsistent data-flow conventions** that will amplify change cost as features grow.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	N/A (no backend)	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	0 backend handlers	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 (external API)	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles (88 files, 137 edges)	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	0 (`utilService` is generic)	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	N/A (no backend)	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (max: `postActions.js` 210 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	N/A (schema external)	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	78.6 avg (58 `.jsx` files)	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	16	MODERATE
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (max: `Message.jsx` 250 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; 71 `useSelector` hooks	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	20	HIGH RISK

H10	Duplicated Domain Logic in Views (additional)	Components duplicating reaction/connection rules	0	1–3	>3	5	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H8	Split cross-domain imports: move comment actions out of `postActions.js`, add `activityFacade` for notification DTO assembly, decentralize socket subscriptions from `Main.jsx`	HIGH RISK	CRITICAL
F5	Migrate React Router v5 (`HashRouter` , `component=` , `useHistory`) to v6 across 20 files; standardize service naming	HIGH RISK	HIGH
F2	Eliminate 16 direct service imports from components — route all reads through Redux thunks and selectors	MODERATE	HIGH
F4	Introduce `AuthContext` + memoized selectors; document single data-access convention	MODERATE	MEDIUM
H10	Extract shared `toggleReaction` and `addConnection` domain helpers; remove duplicated logic from 5 view files	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Bounded feature modules** — user, post, comment, chat, and activity can evolve independently with explicit facades instead of cross-imports.
- **Single data-flow path** — components consume selectors/thunks only, eliminating dual Redux + direct-service paths and stale cache bugs in `userService.usersCash` .
- **Testable domain rules** — extracted reaction/connection helpers become unit-testable without mounting React components.
- **Lower migration cost** — React Router v6 upgrade unlocks modern layouts, error boundaries, and code-splitting patterns.
- **Immutable, predictable state** — normalized Redux store removes embedded comment graphs and direct prop mutation anti-patterns.

---

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by H1 High Cyclomatic Complexity (max CC 41) and H5 Duplicate Code (~12.7%).

EXECUTIVE SUMMARY

This analysis covered **96 frontend source files** across two React applications (`social-media-react` at 87 files, `workbench-demo` at 9 files). No backend/server application layer exists in TARGET_WORKSPACE. Overall code quality is **High Risk**, driven by **high cyclomatic complexity** in notification and messaging components (max CC **41**) and **general duplicate code** estimated at **~12.7%** of normalized frontend LOC. File and class sizes remain within good bounds (largest file **212 LOC**; no files exceed 1000 LOC). Redux action modules repeat identical async thunk boilerplate across four files (**42 instances**). Git churn, defect-prone file, and ownership metrics could not be computed because target application sources are **untracked** in the parent repository history.

2.1 Benchmark Ratings Summary

Manual cyclomatic inspection (branch/loop/ `&&` / `||` / `?:` counting) was used; no ESLint `complexity` rule or Sonar config is present in either project. Duplicate-code percentage derived from normalized 5-line block matching across `social-media-react/src` and `workbench-demo/src`.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	41 (NotificaitonPreview)	HIGH RISK
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	212 (LoginPage.test.tsx / Message.jsx)	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	190 (Signup.jsx `Signup`)	MODERATE
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~8% (Redux thunk + auth-form patterns)	MODERATE
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~12.7% (5-line block analysis)	HIGH RISK
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	n/a (source untracked in git)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	n/a (source untracked in git)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	n/a (source untracked in git)	GOOD
H9	Fragmented useEffect	Max useEffect count per	≤2	3	≥4	3 (NotificaitonPreview.jsx)	MODERATE

	Chains (additional)	component (target ≤2)					

No additional hotspots beyond H9 were observed.

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25% → 50%	88	44.0
Code Churn	25%	n/a	n/a
Defect Density	20%	n/a	n/a
Class/Function Size	15% → 30%	55	16.5
Business Logic Duplication	10% → 20%	75	15.0
Developer Ownership Risk	5%	n/a	n/a
Hotspot Score	**100%**		**76 / 100**

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 High Cyclomatic Complexity	Extract `ActivityMessageStrategy` for notification types; split `useChat` hook in `Message.jsx`; enable ESLint `complexity` rule (threshold 15)	HIGH RISK	CRITICAL
H3 Large Functions	Extract `AuthFormFields` from `Home.jsx` / `Signup.jsx`; decompose `CommentPreview` handlers into service helpers	MODERATE	MEDIUM
H4 Business Logic Duplication	Introduce shared Redux async action factory; consolidate activity copy into `buildActivityMessage()`	MODERATE	MEDIUM
H5 Duplicate Code (general)	Create shared `FormInputGroup` component; add `test-utils.tsx` helpers in workbench-demo	HIGH RISK	HIGH
H9 Fragmented useEffect Chains	Replace 3-effect chains in `NotificaitonPreview.jsx` with `useNotificationPreview` hook or React Query	MODERATE	MEDIUM

2.6 Expected Outcomes

- Lower defect rate in notifications and messaging flows by isolating activity-type logic behind Strategy handlers (CC target <15 per function).
- Faster code reviews and safer refactors as shared form components and Redux thunk factories eliminate ~8–13% duplicated LOC.
- Improved testability via smaller extracted hooks (`useChat*` , `useNotificationPreview`) that can be unit-tested independently of page components.
- Reduced stale-closure and double-fetch bugs from consolidated effect management in notification and search components.

- Future discovery runs can track churn and ownership once frontend projects are committed to git, enabling proactive maintenance of high-change files.

Full report saved to `target/docs/discovery/02-code-quality-complexity.md` (321 lines). Pipeline summary: `agent-runs/20260717T132536_tkr1f1/02-code-quality-complexity-summary.md`.